

# ARM Assembler

и сразу [ссылка](#) на instruction reference

# Регистры

- 64-битные `x1` ,..., `x30`
- В них вложены 32-битные `w1` ,..., `w30`
- `fp = x29` - frame pointer
- `lr = x30` - link register
- `xzr = x31` , `wzr = w31` - zero register
- `sp` - stack pointer
- `pc` - program counter

# Арифметика

- инструкции имеют вид `op dst, src1, src2`
- и выполняются как `dst = src1 <op> src2`
- `add` , `sub`
- `mul` , `smulh` , `umulh` (64-bit) + `smull` , `umull` (32-bit)
- `sdiv` , `udiv`
- `and` , `orr` (or), `eor` (exclusive or), `mvn` (not)

# Поток управления

- `b label` - branch
- регистр флагов - `pstate = pr = x18`
- `subs = cmp`, `ands = tst` - выставляет флаги
- `b<cond> label` или `b.<cond> label`
  - `eq`, `ne`
  - `lt`, `le`, `gt`, `ge`
  - `lo`, `ls`, `hi`, `hs`

# Работа с памятью

- `ldr Rt, <mem>`, `str Rt, <mem>`, где `<mem>` это:
  - `label`
  - `[Rt]` - `*Rt`
  - `[Rt, pimm]` - `*(Rt + pimm)`
  - `[Rn, Rm]` - `*(Rn + Rm)`
  - `[Rn, Rm, lsl amount]` - `*(Rn + (Rm << amount))`
  - `[Rt, simm]!` - pre-index (похоже на `++Rt`)
  - `[Rt], simm` - post-index (похоже на `Rt++`)

# Работа с памятью

- размер операнда - задается суффиксом (напр. `strb`)
  - `b` - byte
  - `h` - half-word
  - `w` - word
  - иначе - по размеру регистра
- `adr label` - вместо `lea`

# Соглашение о вызовах

- `bl label` (branch with link) + `ret`
- "link" - сохраняется в регистр `lr`
- caller-saved:
  - `x0-x7` - аргументы
  - `x0` - возвращаемое значение
  - `x8` , `x9-x15` , `x16-x17` , `x18`
- callee-saved
  - `x19-x28`
  - `fp` , `lr` , `sp`

# RISC vs CISC

- Reduced vs Complex [Instruction Set Computer]
- фиксированная длина кодирования против переменной
- RISC
  - инструкций меньше, выполняются быстрее
  - архитектура процессора проще, энергоэффективность выше
  - нет сложных инструкций
  - побочный эффект: нет простых инструкций
    - `add` умеет опционально скастовать / сдвинуть второй аргумент
    - `mul` - это `madd`
    - `mvn` - это `orn`